

Course No. CSSE-601

Distributed Computing

Course No. CSSE-601

Type of Distributed System Design Goals

Objective

- Introduction
- Middleware & Distributed Systems
- Design Goals
 - Supporting Resource Sharing.
 - Distribution transparent.
 - Openness.
 - Scalability.
- Types of distributed systems.
 - Cluster Computing.
 - Grid Computing.
 - Cloud Computing.
- Distributed Information Systems.

Introduction

- **What is a distributed system?**
- A **distributed system** is a **collection of autonomous computing elements** that **appears** to its users as a **single coherent system**.
- It refers **two characteristic** features of **distributed systems**.
- **Characteristic 1:** Collection of **autonomous computing elements**.
- **Characteristic 2:** Single coherent system.

Introduction

- **Characteristic 1: Modern distributed systems** can, and often will, consist of **all kinds of nodes**, ranging from very big **high-performance** computers to small plug **computers** or even **smaller devices**.
- **Nodes** are programmed to **achieve common goals**, which are realized by **exchanging** messages with each other.
- An important **observation** is that, as a consequence of **dealing** with **independent nodes**, each one will have its own notion of time (**Global Clock**).
- **Collection of nodes** implies that we may also need to **manage** the **membership** and **organization** of that **collection**.

Introduction

- **Characteristic 1:**
- **Managing group membership** can be exceedingly **difficult**, if only for reasons of admission control. (**Open** and **Closed** groups).
- **Open Group:** **Any node** is **allowed** to **join** the **distributed system**, **effectively** meaning that it can **send messages** to any **other node** in the **system**.
- **Closed Group:** **only the members** of that **group** can **communicate** with **each other** and a **separate mechanism** is **needed** to let a **node** **join** or **leave** the **group**.

Introduction

- **Characteristic 1:**
- **Structured overlay:** In this case, each node has a **well-defined set of neighbors** with whom it can **communicate**. For example, the nodes are **organized** in a **tree** or **logical ring**.
- **Unstructured overlay:** In these **overlays**, each **node** has a number of **references** to **randomly selected other nodes**.

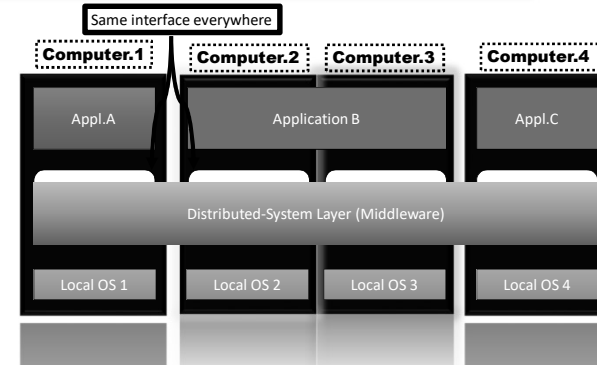
Introduction

- **Characteristic 2:**
- A **distributed system** should **appear** as a **single coherent system**.
- Offering a **single coherent** view is **often challenging enough**.
- It requires that an **end user** would not be able to tell **exactly** on which **computer** a **process** is **currently executing**, or even perhaps that part of a task has been **spawned off** to **another process** **executing somewhere** else. (**Distribution Transparency**)

Middleware & Distributed Systems

- To assist the **development of distributed applications**, distributed systems are often **organized** to have a **separate layer of software** that is **logically placed on top** of the respective **operating systems** of the **computers** that are **part of the system**.
- A **manager of resources** offering its **applications** to **efficiently share** and **deploy** those **resources across a network**.

Middleware & Distributed Systems



Middleware & Distributed Systems

- **Few examples of typical middleware services.**
- **Communication:** RPC.
- **Transactions:** Atomic Transactions.
- **Service composition:** Web Services
- **Reliability:** Horus toolkit.

Design Goals

- **Four important goals** that should be met to **make building a distributed system** worth the effort.
- Supporting resource sharing.
- Distribution transparent.
- Openness.
- Scalability.

Supporting Resource Sharing.

- **Resources** can be virtually anything, but typical examples include **peripherals, storage facilities, data, files, services, and networks.**
- Many reasons for **wanting to share resources.** For example **Cheaper, Collaborate, Exchange Information, Software-Development, Software Updates, Backup Services, Data Synchronization.**

Distribution Transparent.

- **Distributed system** is to **hide** the fact that its **processes** and **resources** are **physically distributed** across **multiple computers** possibly separated by large distances.

Distribution Transparent.

Transparency	Description
Access	Hide differences in data representation and how an object is accessed
Location	Hide where an object is located
Relocation	Hide that an object may be moved to another location while in use
Migration	Hide that an object may move to another location
Replication	Hide that an object is replicated
Concurrency	Hide that an object may be shared by several independent users
Failure	Hide the failure and recovery of an object

Distribution Transparent.

- **Degree of Distributed Transparency:** Trade-off between a **high degree of transparency** and the **performance** of a **system.**
- Many **Internet applications** repeatedly try to **contact a server** before finally giving up. Consequently, **attempting** to mask a **transient server failure** before trying another one may **slow down the system** as a whole.

Openness

- It is **essentially** a **system** that offers **components** that can **easily be used** by, or **integrated** into other **systems**.
- An **open distributed system** itself will often consist of **components** that **originate** from **elsewhere**.
- **Interoperability**, **composability**, and **extensibility**.
- **Standard rules** that describe the **syntax** and **semantics** of what those **components** have to offer.

Openness

- A **general approach** is to define **services** through **interfaces** using an **Interface Definition Language (IDL)**.
- **Interoperability** characterizes the **extent** by which **two implementations** of **systems** or **components** from **different manufacturers** can **co-exist** and **work together** by merely relying on each **other's services** as specified by a **common standard**.

Openness

- **Portability** characterizes to what extent an **application developed** for a **distributed system A** can be **executed**, without **modification**, on a different **distributed system B** that **implements** the same **interfaces as A**.
- It should be easy to **configure** the **system** out of different **components**. it should be easy to **add new components** or **replace existing** ones without **affecting** those **components** that stay in place (**Extensible**).

Scalability

- **Worldwide connectivity** through the **Internet** is as **common** as being able to **send a postcard to anyone anywhere** around the **world**.
- With this in mind, **scalability** has become one of the **most important design goals** for **developers of distributed systems**.

Scalability

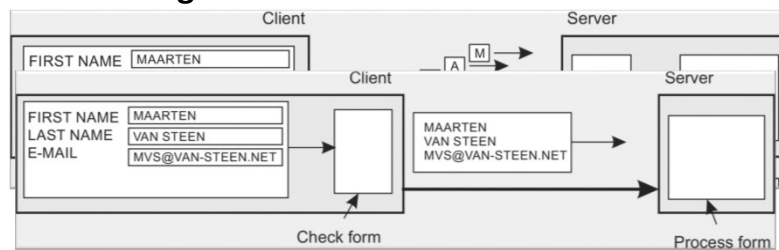
- Scalability Dimensions:
- **Size Scalability.**
 - Easily add more users and resources to the system without any noticeable loss of performance.
- **Geographical Scalability.**
 - Communication delays may be significant.
- **Administrative Scalability.**
 - Many independent administrative organizations.

Scalability

- Scalability Techniques:
 - **Hiding Communication Latencies.**
 - **Distribution of Work**
 - **Replication**

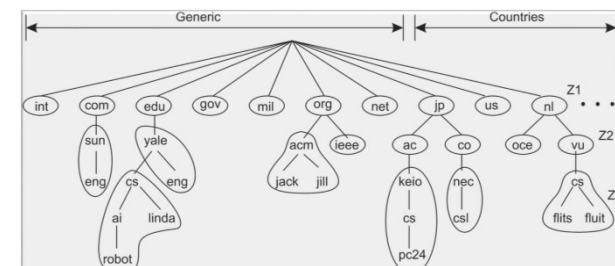
Scalability

- Scalability Techniques:
 - **Hiding Communication Latencies.**



Scalability

- Scalability Techniques:
 - **Distribution of Work**



Scalability

Scalability Techniques:

- **Replication:** Replication not only increases **availability**, but also helps to **balance the load** between **components** leading to **better performance**.
- **Caching** is a **special form of replication**, although the **distinction** between the **two** is often **hard to make** or **even artificial**.
- There is one **serious drawback** to **caching** and **replication**. (**Consistency**)

Types of Distributed Systems

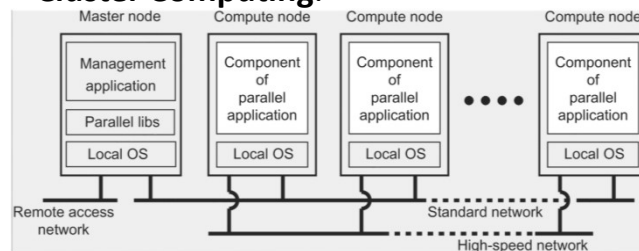
•High performance distributed computing.

- Cluster Computing.
- Grid Computing.
- Cloud Computing.

Types of Distributed Systems

•High performance distributed computing.

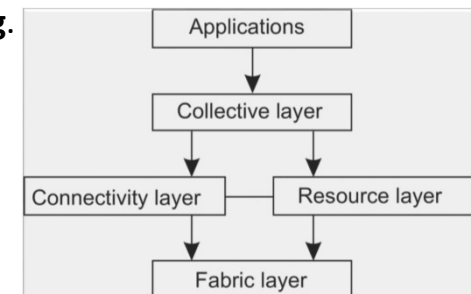
•Cluster Computing.



Types of Distributed Systems

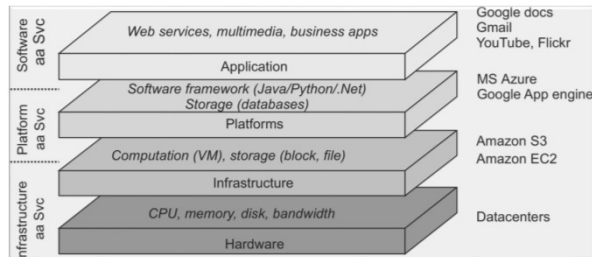
•High performance distributed computing.

•Grid Computing.



Types of Distributed Systems

- High performance distributed computing.
- **Cloud Computing.**



Distributed Information Systems

- **Networked application** simply consists of a **server** running that **application** and **making it available** to **remote programs**, called **clients**.
- Such **clients** send a **request** to the **server** for **executing** a specific **operation**, after which a **response** is **sent** back.
- **Integration** at the lowest level allows **clients** to wrap a **number of requests**, possibly for **different servers**, into a **single larger request** and have it **executed** as a **distributed transaction**.

Distributed Information Systems

- As **applications** became **more sophisticated** and were **gradually separated** into **independent components**.
- it became clear that **integration** should also take place by **letting applications communicate** directly with **each other**.
- This has now lead to a **huge industry** that **concentrates** on **Enterprise Application Integration (EAI)**.

Query Regarding Lecture

Email: dr. m.naseem622@gmail.com

Thanks

Email: dr.m.naseem622@gmail.com